

EC 220: Computer System Architecture

- **Instructor**
- Lt Col Hammad Tanveer Butt
- E-mail:
- **Credits**
- 3+1 (Lec + Lab)
- Lec : 3 hrs/wk (Meet Twice a Week 1.5 hrs)
- Lab: 3 hrs/wk (Once a week)
- Home Study: 7hrs/wk
- **Prerequisites**
- EC 210: Logic and Sequential Circuit Design
- EC 120: Computer Organization
- **Required for**
- EC 310: Microcontroller and Microprocessors

Grading

Quizzes:	20%
Assignments:	30%
Sessional Exam:	20%
Final Exam:	30%
Lab Exam	20%
Lab Project	80%

EC 220: Computer System Architecture

- **What will you Learn in this Course ?**
- **Self Study Learning:**
 - Unsigned and Signed Binary Number Representations
 - Computer Arithmetic
 - Floating Point Binary Representations (Single/Double Precision)
 - Arithmetic Logic Unit Implementation from MSI Logic
- **Lectures:**
 - As per Course Outline
- **Lab:**
 - Evaluate and Understand Performance of different Computer Systems
 - Implement Algorithms in Assembly Language (x86 or MIPS)
- **Project:**
 - Run an Embedded System and obtain data on Linux Host via Bluetooth. Also access Memory content on embedded device
 - Run Android App to obtain inertial sensors data on Linux Host via TCP/router and evaluate latency of data arrival and packet loss

EC 220: Computer System Architecture

- **Things you'll be learning (Broad View):**
 - how computers work, a basic foundation
 - how to analyze their performance (or how not to!)
 - issues affecting modern processors (caches, pipelines)
- **Why learn this stuff?**
 - you want to call yourself a “computer/software engineer”
 - you want to build software people use (need performance)
 - you need to make a purchasing decision or offer advice

EC 220: Computer System Architecture

Topics covered in the Course and Level of Coverage	
• Introduction to Computer System Architecture and Digital Computers.	3hrs
▪ Instruction set, Representation of instructions in machine language	3hrs
▪ Addressing modes, Signed, Unsigned numbers binary representation, Arithmetic operations, Overflow detection	3hrs
▪ Multiplication and Division algorithms and associated hardware	3hrs
▪ Floating point numbers binary representation (Single / Double Precision), IEEE 754 Standard, Floating point Arithmetic operations (Addition, Multiplication Algorithms)	3hrs
▪ Assessing and understanding performance, CPU performance and its factors	3hrs
▪ Design of Arithmetic Logic Unit and Control Unit. Building of single cycle processor (Datapath And Control)	3hrs
▪ Multi-Cycle processor implementation (Datapath And Control), FSM And Exceptions Handling	3hrs
▪ Enhancing Performance with Pipelining, Overview of Pipelining, Pipelined Datapath and Control	3hrs
▪ Data Hazards, Structural Hazards, Branch hazards, Forwarding/Bypassing, Stalling, Branch Prediction Techniques	3hrs
▪ Memory system design, memory hierarchy, primary memory (RAM, ROM), secondary memory. Function table of RAM & ROM, memory address map. Interfacing microprocessor with memory.Memory cycle, memory read cycle, memory-write cycle.	3hrs
▪ Cache Memory & its Mapping techniques, Cache Write and Data Replacement Policies, Different designs of cache memory system.	3hrs
▪ Stack operations, Subroutines, Interrupts, Priority Interrupt.Virtual Memory, Paging.Introduction to parallel processing, Multiprocessor systems, Pipeline processing.	3hrs
▪ I/O Architectures & Data Transmissions Modes memory mapped input/output, isolated input/output. Introduction toParallel and Multiprocessor Architectures	3hrs

EC 220: Computer System Architecture

Course book and Related Course Material

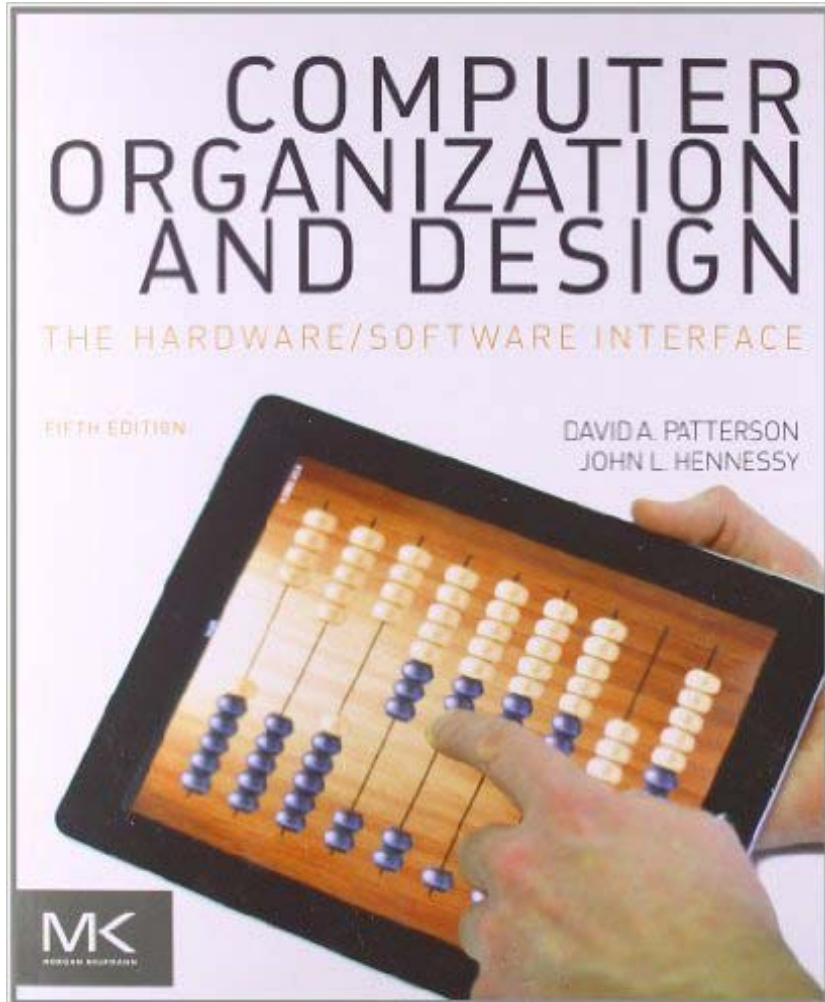
Textbooks:

1. Computer Organization and Design – The Hardware / Software Interface, 5th Edition, by David A. Patterson & John L. Hennessy.

Reference Books:

1. Computer Architecture- A Quantitative Approach, Fifth Edition by John Hennessy, David Patterson, 2011
2. Computer Organization and Architecture: Designing for Performance, 8th Edition, 2010 by William Stallings.

Textbook



Textbook: “P&H”

***Computer Organization
and Design***

Patterson and Hennessy

Fifth Edition

Morgan Kaufmann

© 2014 (MIPS)

© 2017 (ARM)

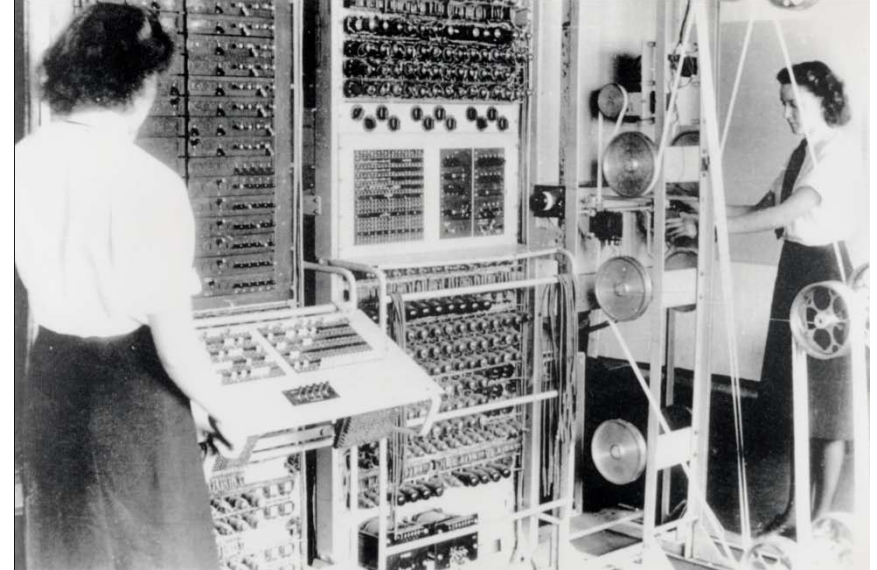
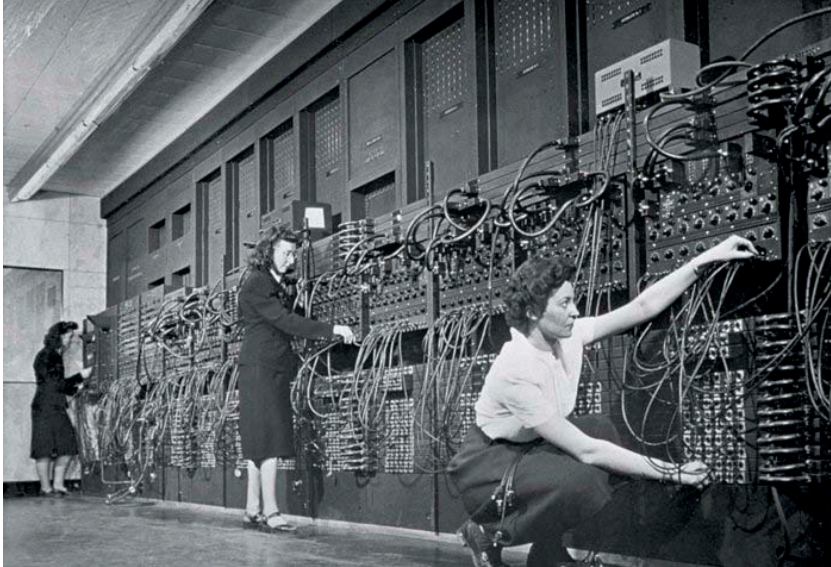
Book Contents

- Performance issues (Ch 1 – P&H 5th Edition)
- A specific instruction set architecture (Ch 2)
- Arithmetic and how to build an ALU (Ch 3)
- Constructing a processor to execute our instructions (Ch 4)
- Pipelining to improve performance (Ch 4)
- Caches, main, and virtual memory, I/O (Ch 5)
- Multiprocessor Architectures (Ch 6)
- Future Computing Technologies (instructor)

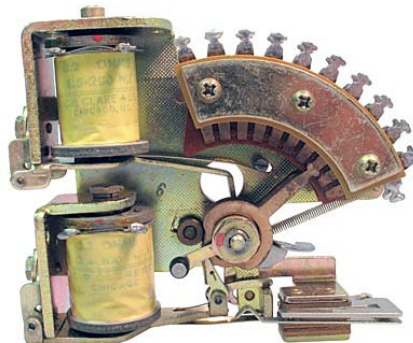
Mechanical Computer Architecture



Wired vs Reprogrammable Digital Computer

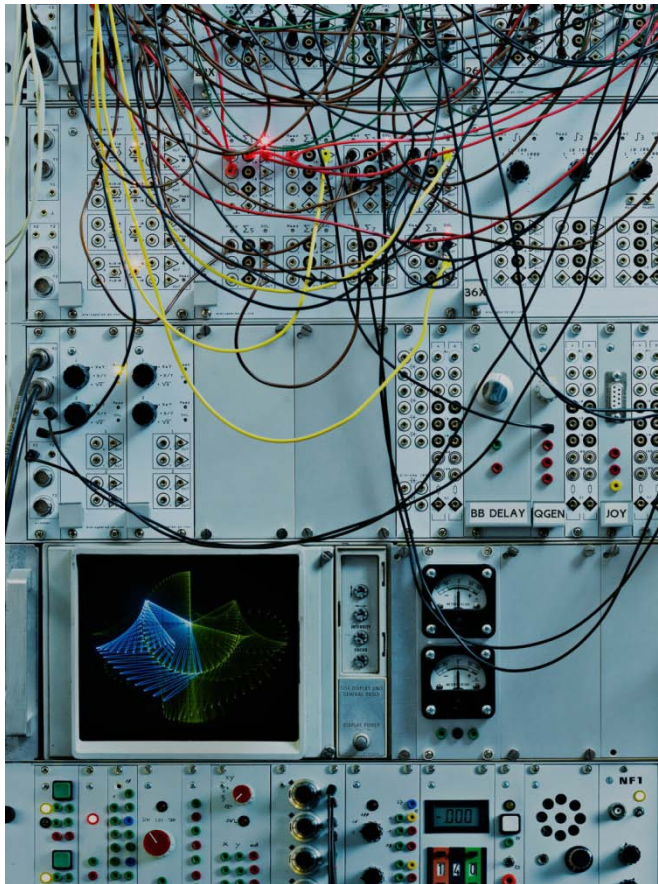


Software/ Hardware
Tradeoff
Co-Design



FPGA?
vs ASIC / SoC

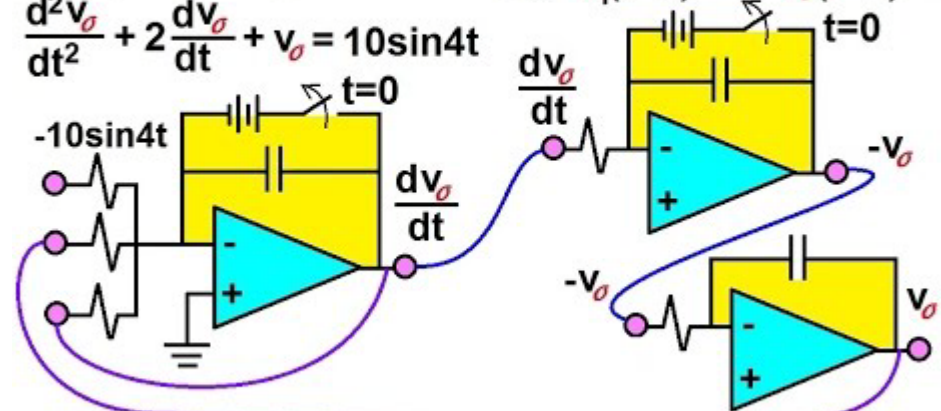
Wired Analog Computer



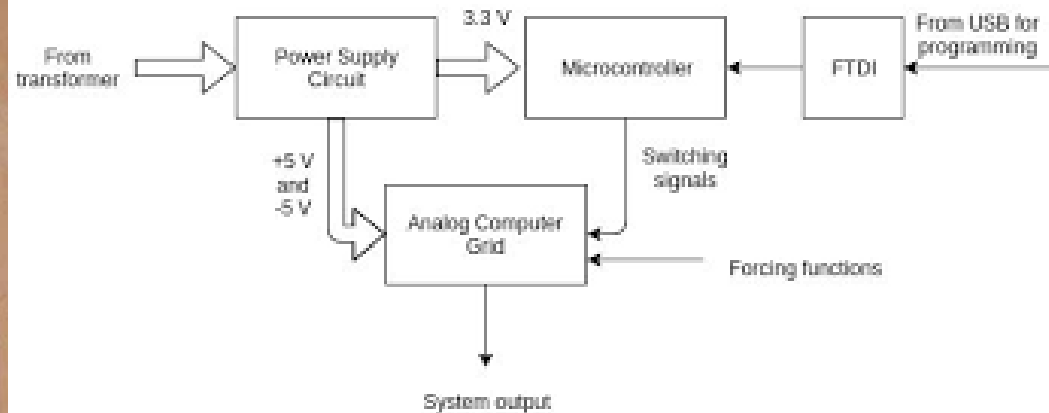
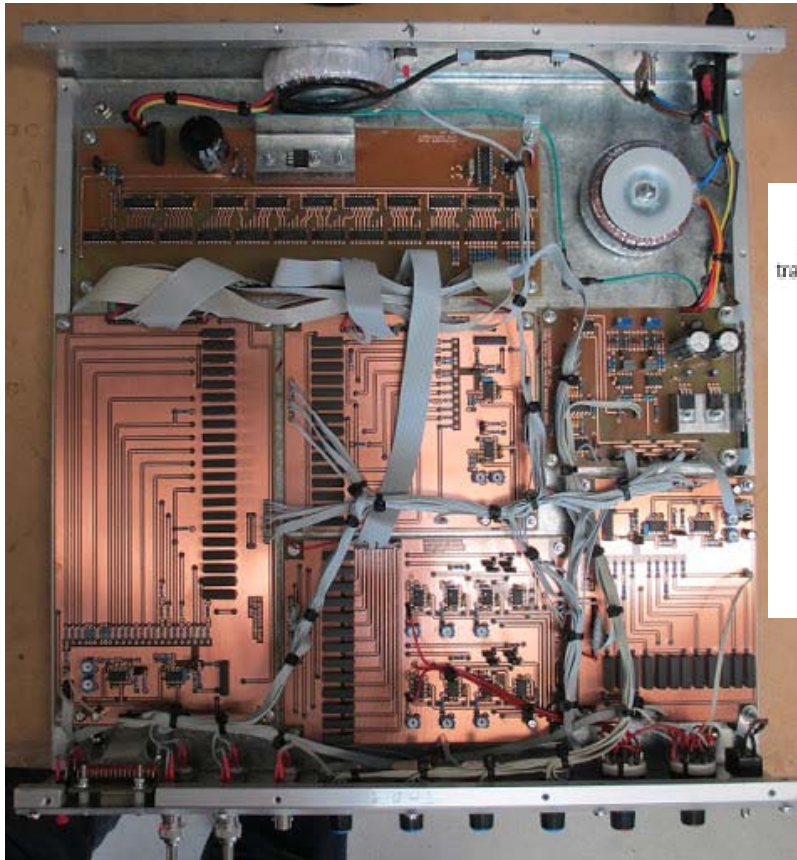
The Analog Computer: Example

Design a computer for: $t > 0 \quad v_{o1}(t=0) = -4 \quad v_o(t=0) = 1$

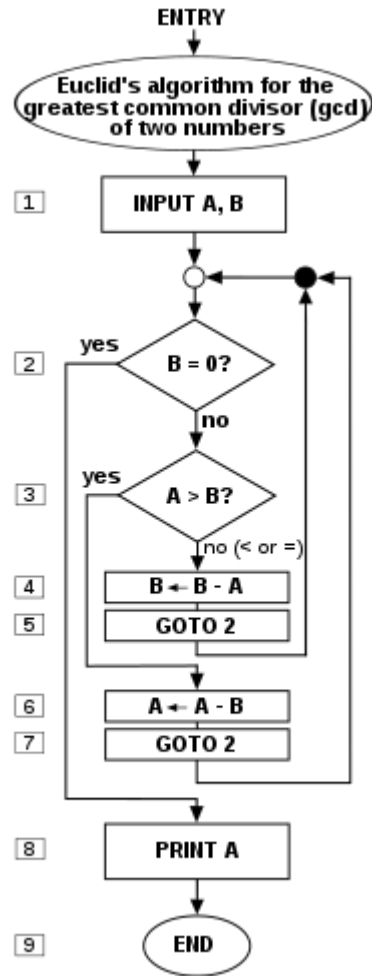
$$\frac{d^2 v_o}{dt^2} + 2 \frac{dv_o}{dt} + v_o = 10 \sin 4t$$



Reprogrammable Analog Computer

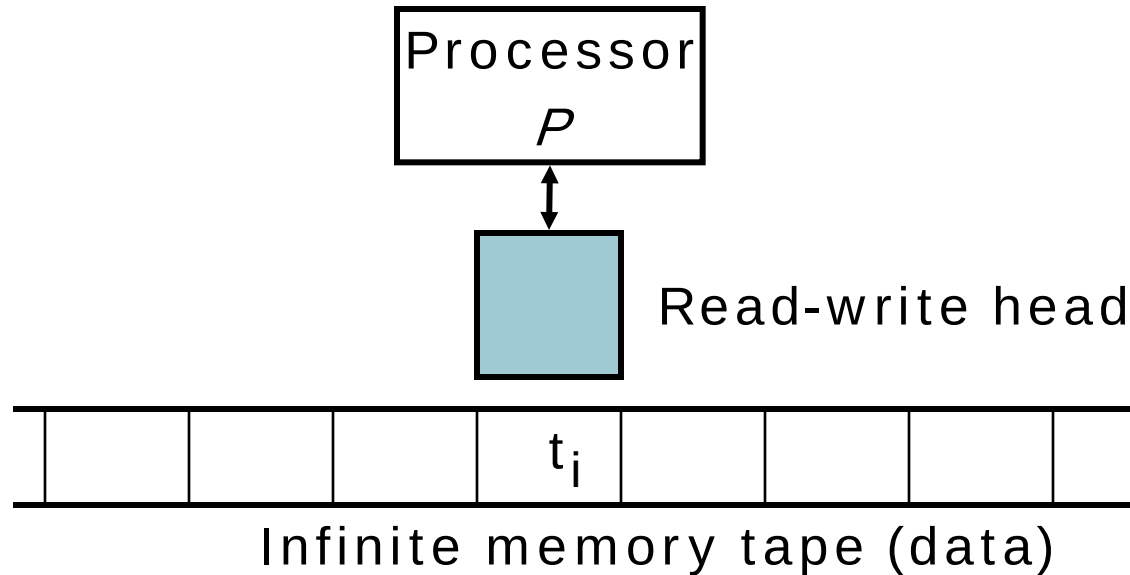


Algorithm – Discrete Sequence of Steps



Assembly Line

Turing Machine



Machine instruction: s_h t_i o_j

s_k

Present state of processor

Symbol on tape

Operation

Next state of processor

Turing Machine

- Four operations:
 - $o_j = t_j$, replace present symbol t_i by t_j
 - $o_j = R$, move head one position to right
 - $o_j = L$, move head one position to left
 - $o_j = H$, halt the computation
- Universal Turing Machine: small instruction set, $\#symbols \times \#states < 30$; can perform any possible (*computable*) computation.
- *Computable* means that Turing machine halts in finite number of steps.
- Real computers have finite memory – they find certain problems *intractable*.

Example

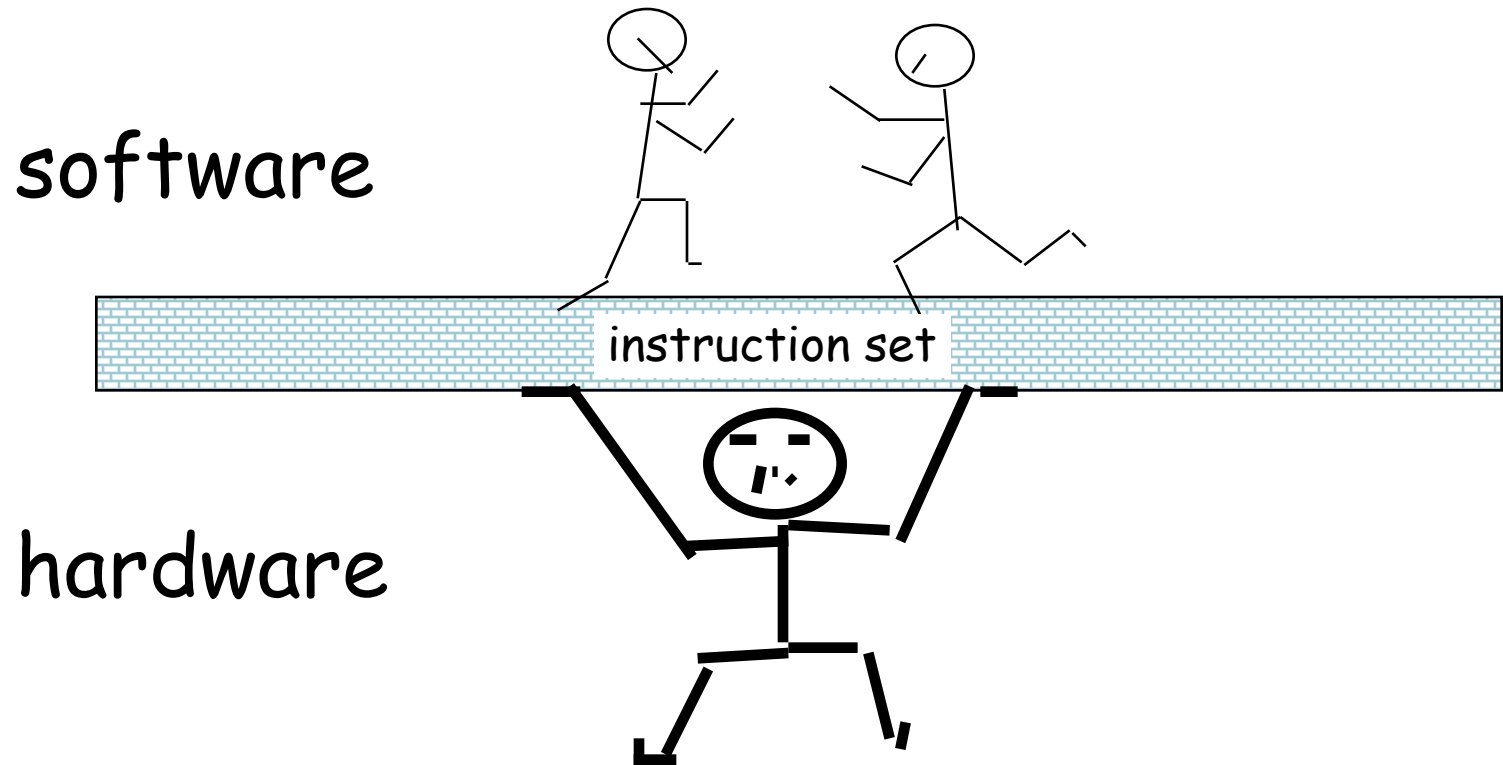
- Start with a blank tape and create a pattern 0b1b0b1b . . .
- Define symbols: b (blank), 0, 1

Present state	Symbol on tape	Operation	Next state
S0 (begin)	blank	Write 0 and move right	S1
S1	blank	Move right	S2
S2	blank	Write 1 and move right	S3
S3	blank	Move right	S0

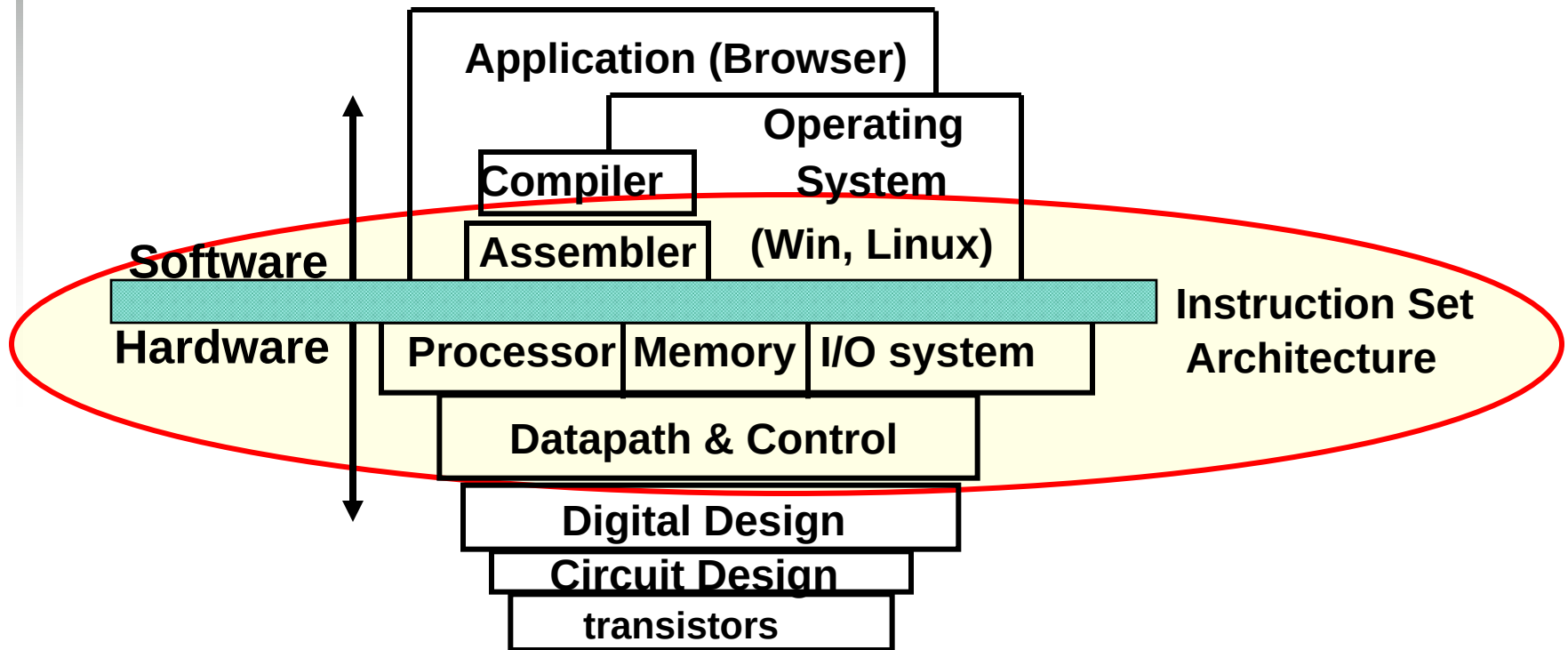
What is *Computer Architecture*

Computer Architecture =
Instruction Set Architecture +
Organization + Hardware + ...

The Instruction Set: a Critical Interface



Levels of Abstraction

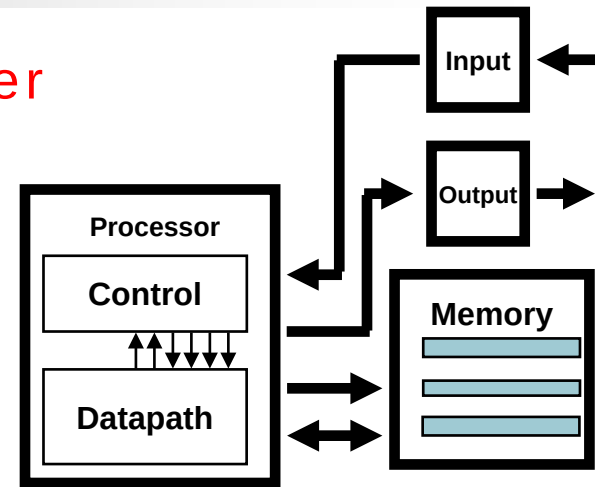


- Coordination of many *levels of abstraction*

Big Ideas

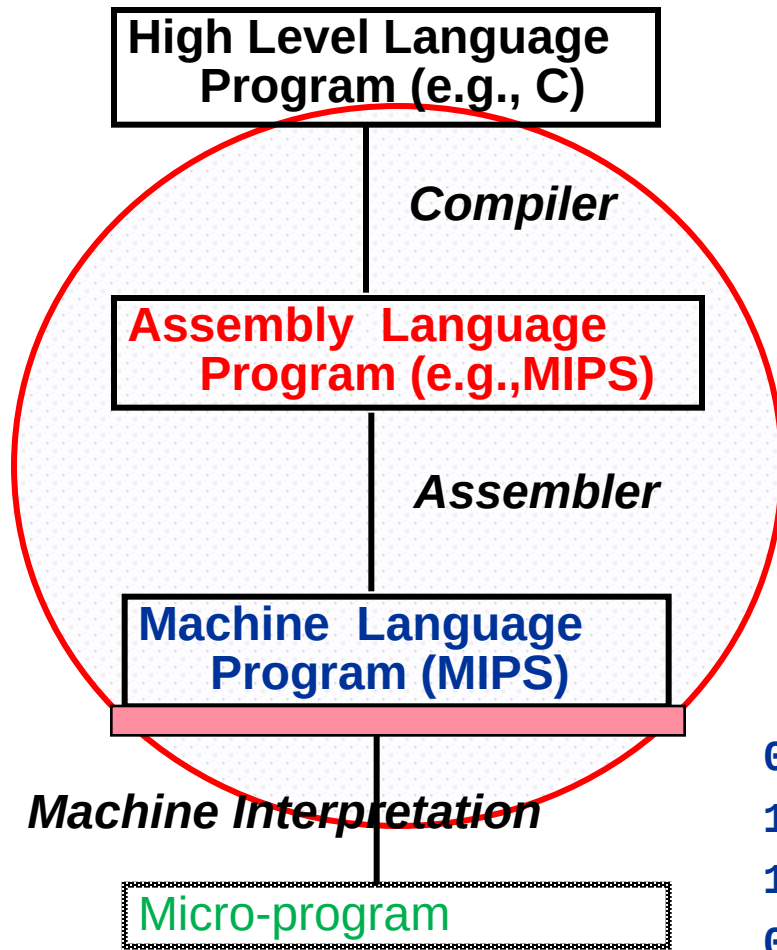
- 5 Classic components of a Computer

- Memory
- CPU
- I/O
- Bus
- Clock



- **Data** can be anything (integers, floating point, characters): a program determines what it is
- **Stored program** : instructions just data
- **Principle of Locality**, exploited via a memory hierarchy (cache and virtual memory)
- Greater performance by exploiting **parallelism**
- **Principle of abstraction**, used to build complex systems as layers

Levels of Representation



```
temp = v[k];  
v[k] = v[k+1];  
v[k+1] = temp;
```

```
lw      $t0, 0($2)
```

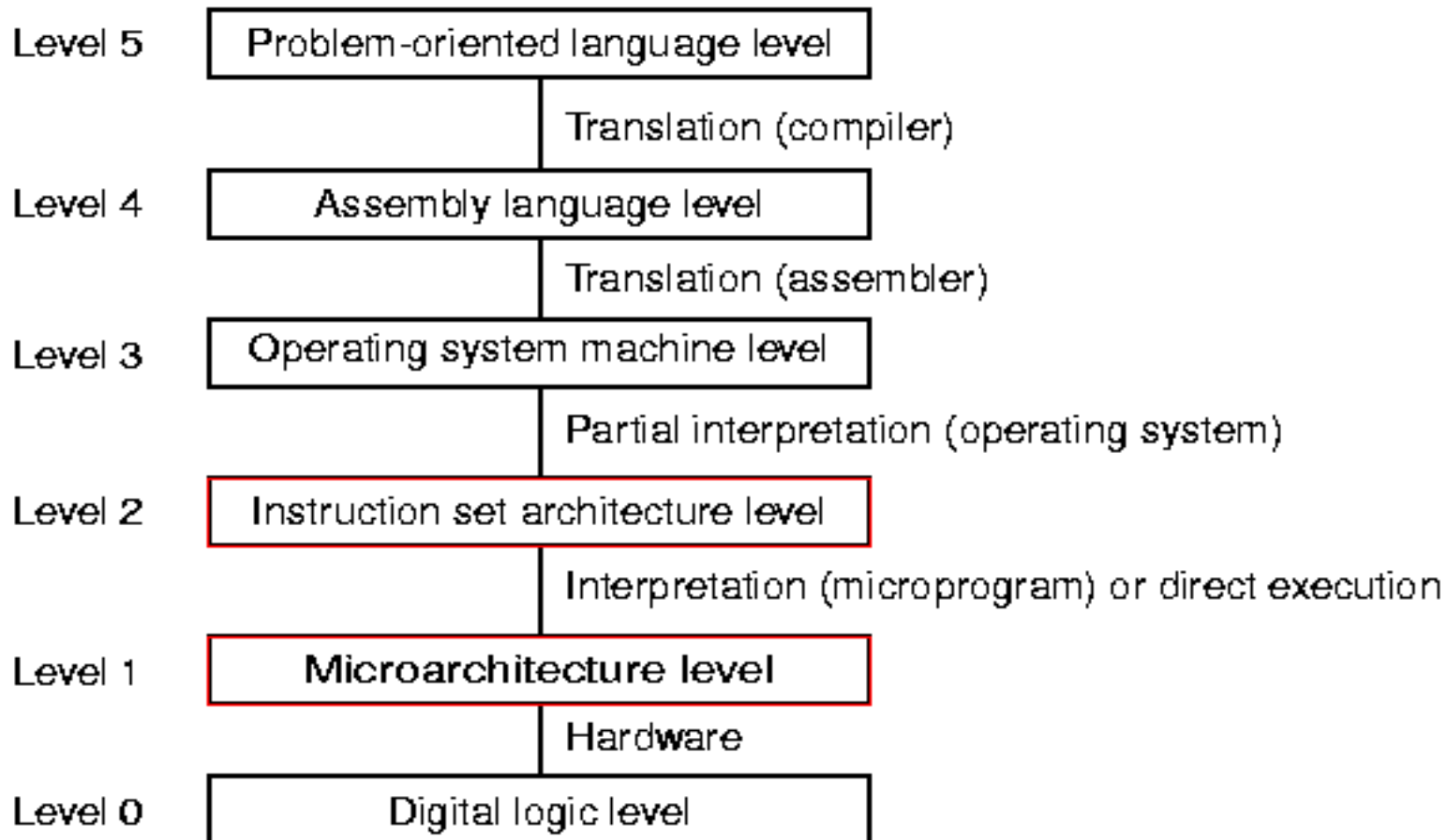
```
lw      $t1, 4($2)
```

```
sw      $t1, 0($2)
```

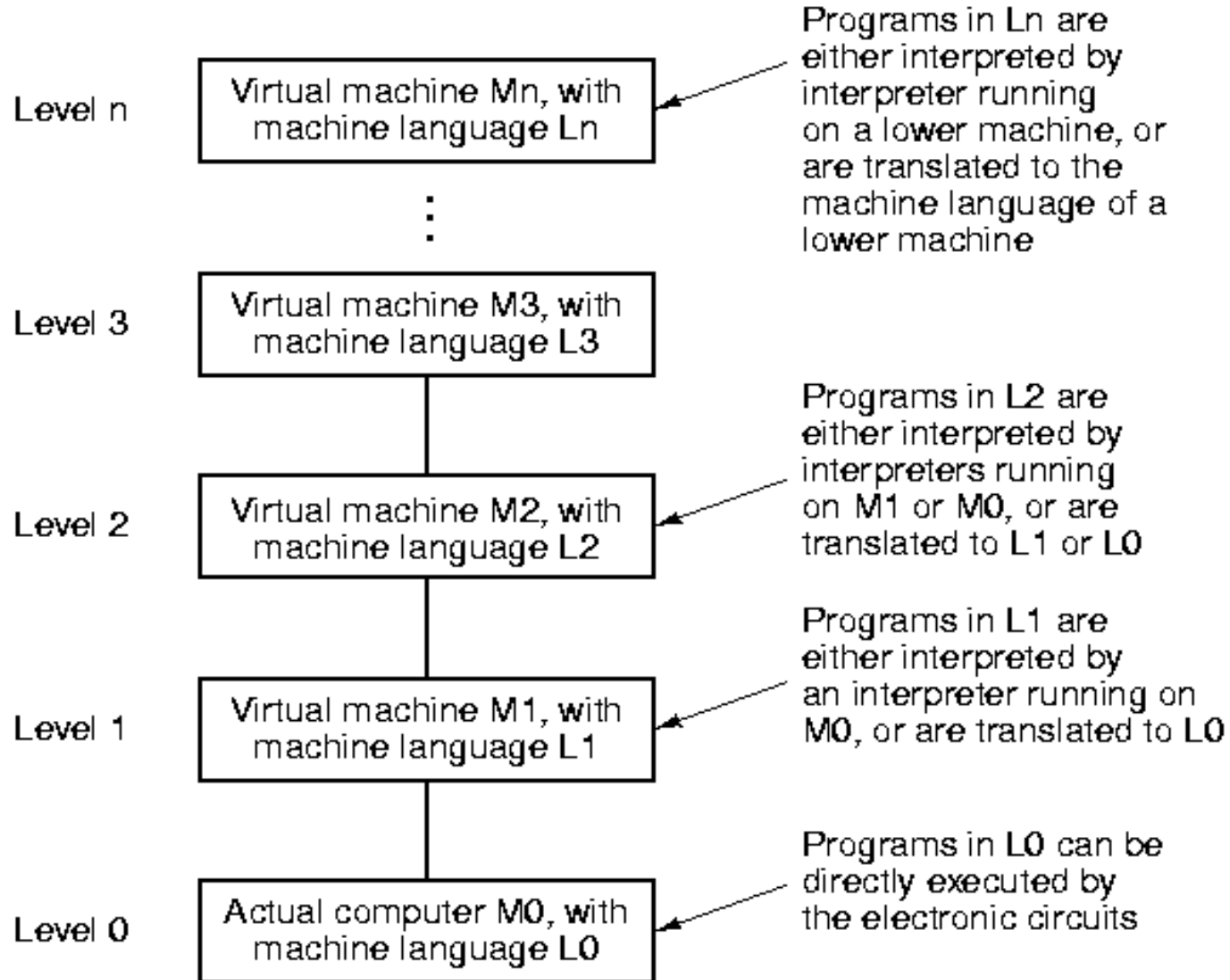
```
sw      $t0, 4($2)
```

```
0000 1001 1100 0110 1010 1111 0101 1000  
1010 1111 0101 1000 0000 1001 1100 0110  
1100 0110 1010 1111 0101 1000 0000 1001  
0101 1000 0000 1001 1100 0110 1010 1111
```

Six-Level Abstraction



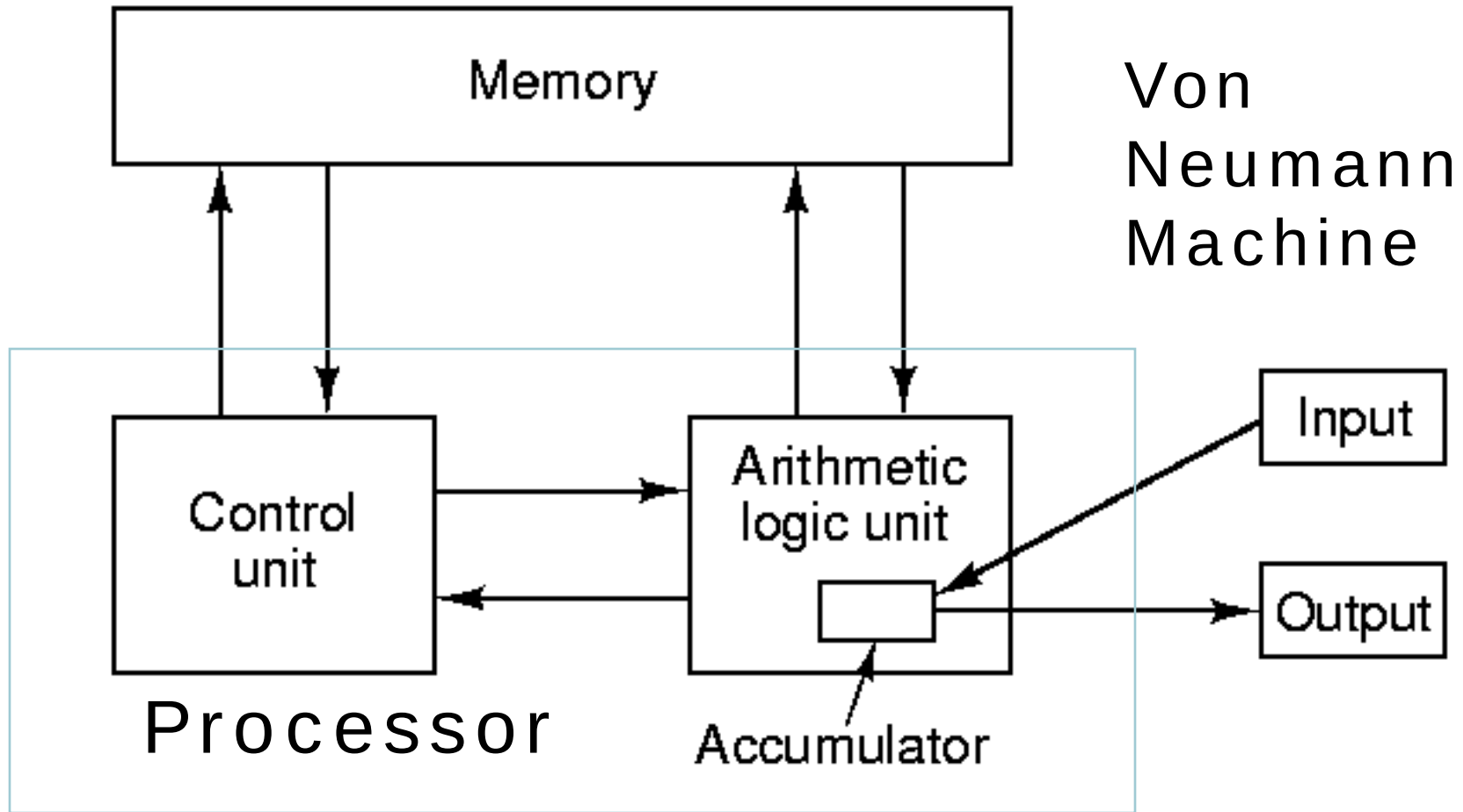
Virtual Machine



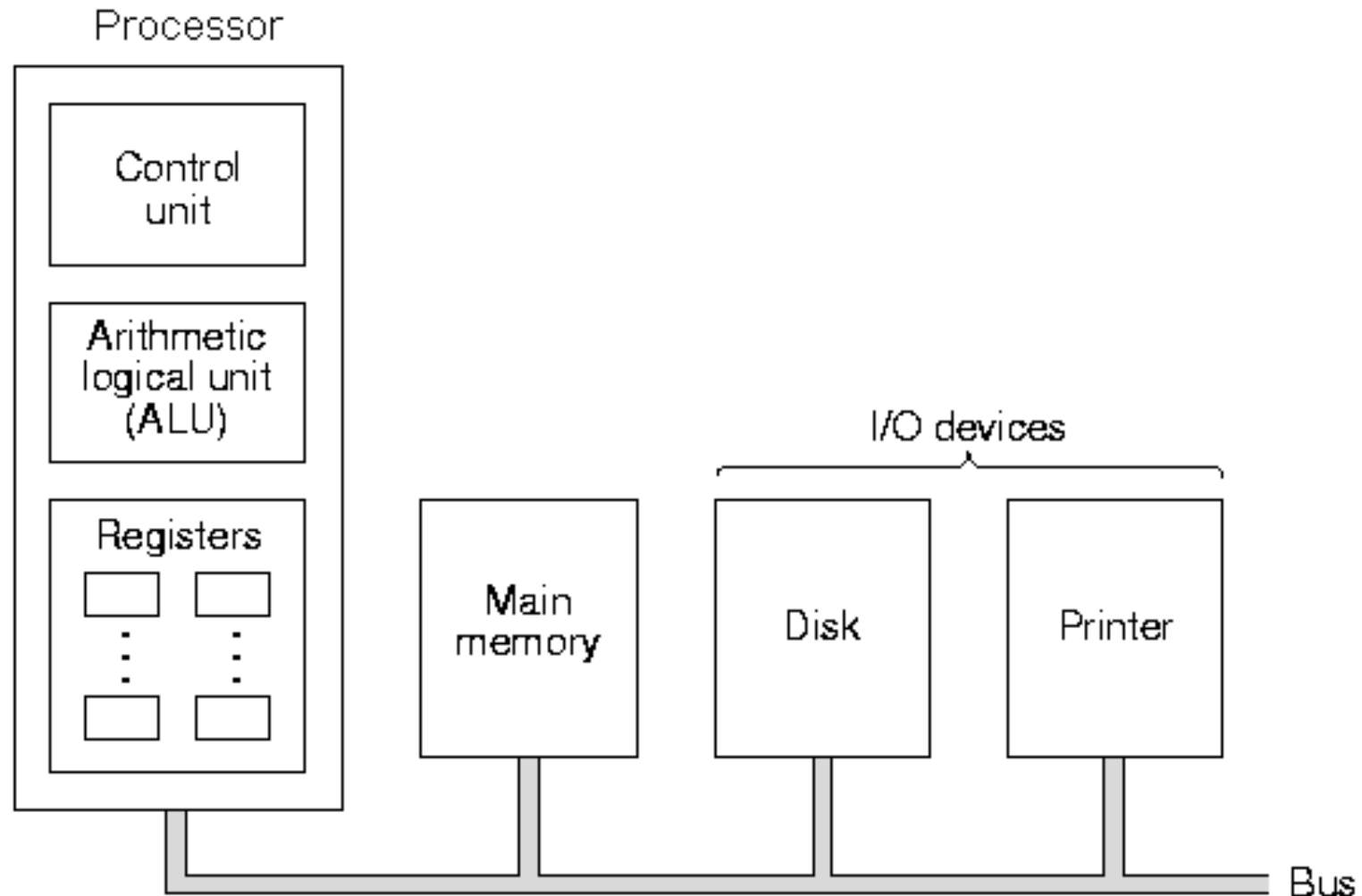
Design Principles

- CISC vs. RISC (VLIW)
- Instructions directly executed by hardware
- Maximize instruction issue rate (ILP)
- Simple instructions (easy to decode)
- Access to memory only via load/store
- Plenty of registers
- Pipelining

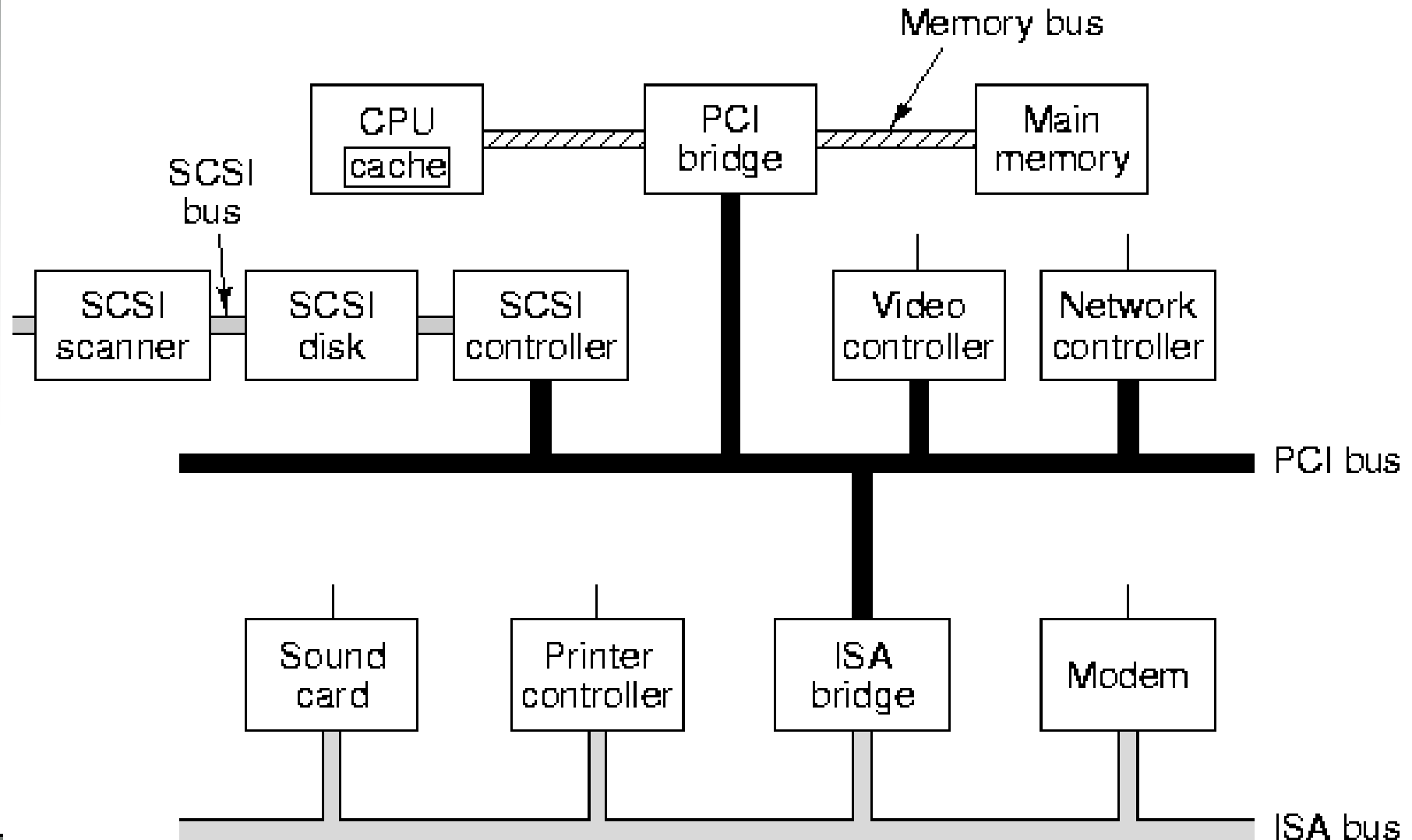
Computer Organization



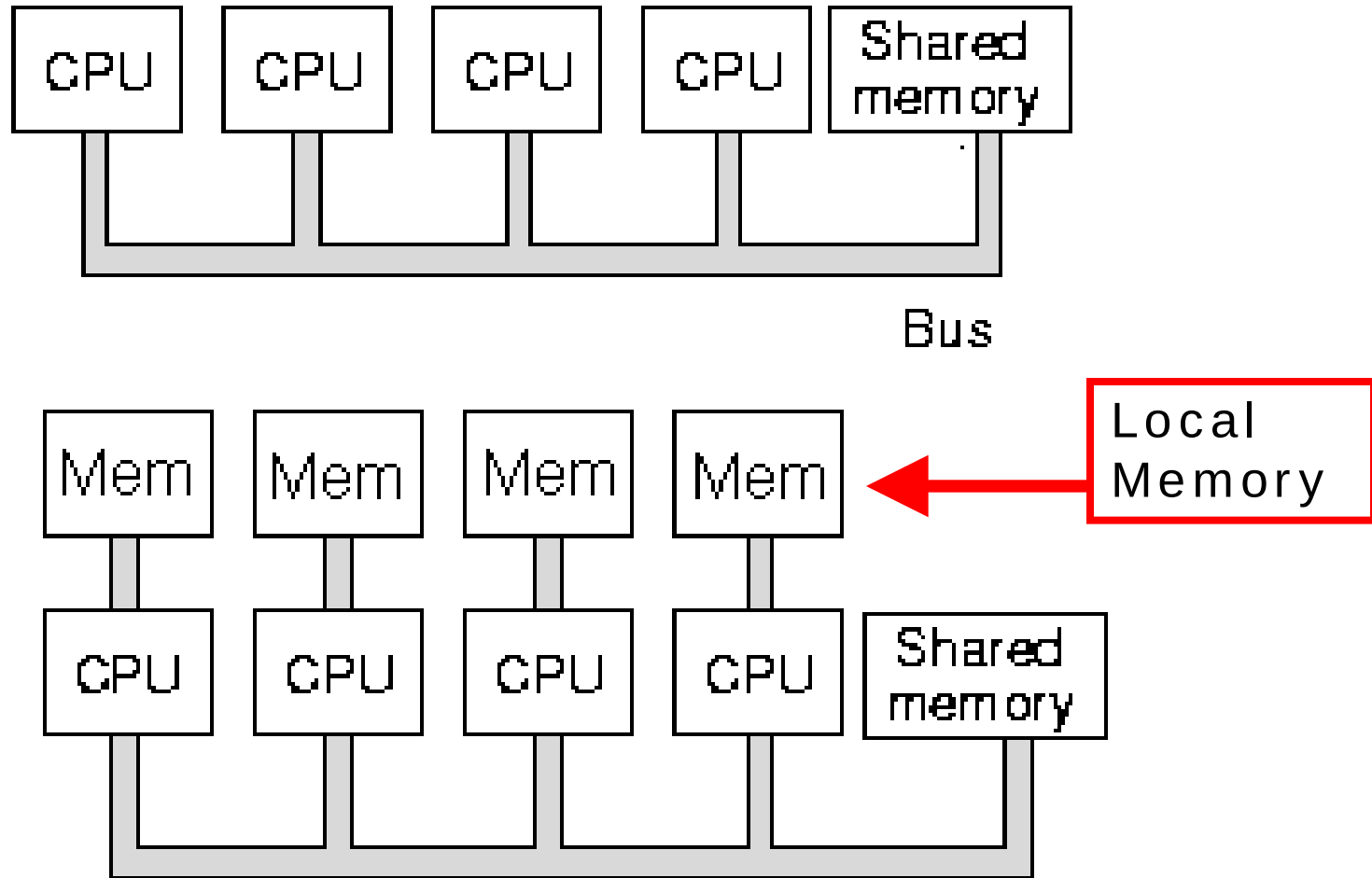
Bus-Based Computer



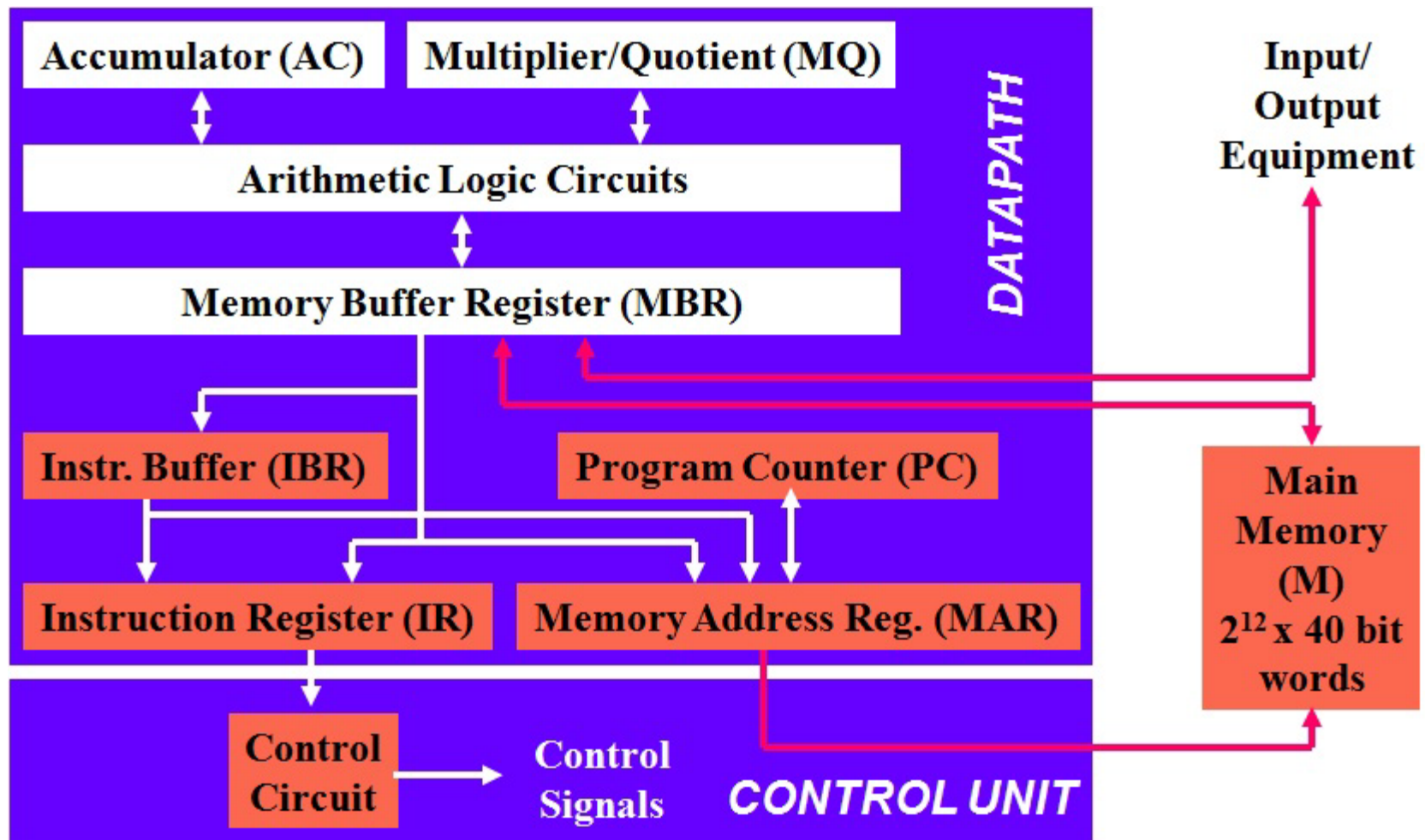
Classic PC Architecture



Multiprocessors

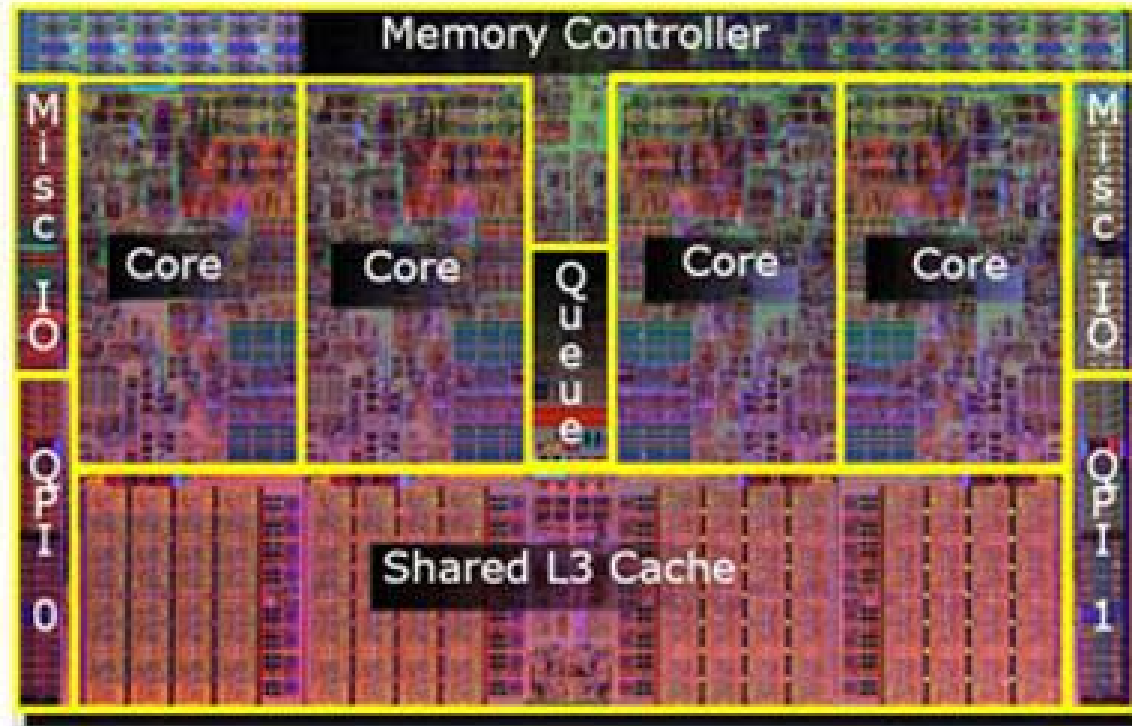


Executing the Program



Micro-architecture

The First Nehalem Processor



QPI: Intel® QuickPath Interconnect

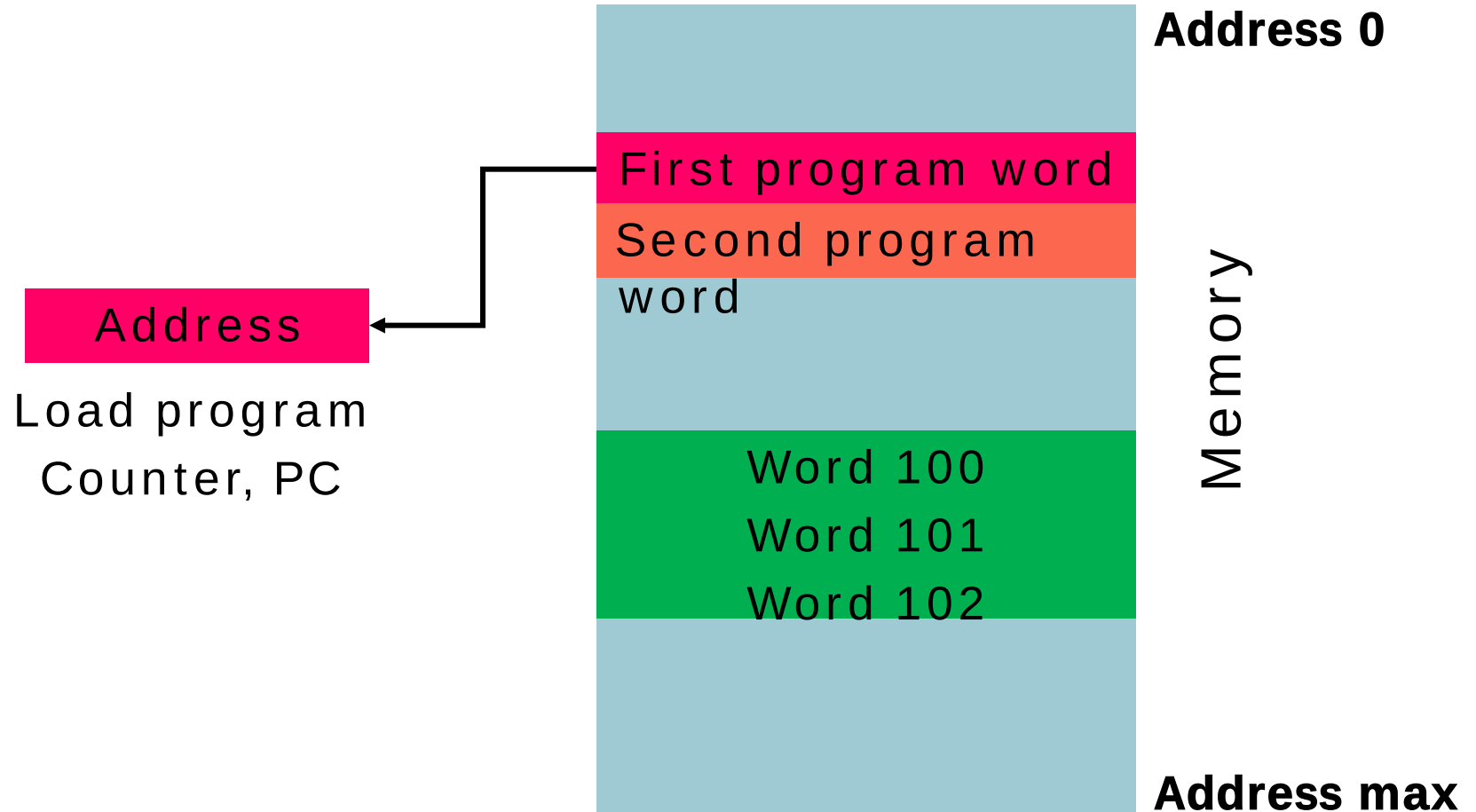
Intel Developer
FORUM

A Modular Design for Flexibility

Nehalem: Next Generation Intel® Microarchitecture 1



Save Program in Memory



Datapath (Microarchitecture)

